

UNIVERSITÉ MOHAMMED PREMIER
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES D'OUJDA
Filière Génie Informatique – GI3

Rapport de Mini-Projet

MedVision

Application JavaFX de gestion d'hôpital

*« Là où la rigueur technique rencontre l'humanité du soin,
l'interface devient un langage clair au service de la vie. »*

Réalisé par :

Demnati Ismail
Salhi Amine

Encadrantes :

Mme Ilhame El Farissi
Mme Douae El Hila

Module : Développement Java IHM
Année universitaire : 2025/2026

Remerciements

Nous tenons tout d'abord à exprimer nos sincères remerciements à Mme Douae El Hila, pour son accompagnement, ses orientations et les consignes claires données dans le cadre du mini-projet du module Développement Java IHM.

Nous adressons également nos sincères remerciements à Mme Ilhame El Farissi pour son accompagnement, ses conseils et son encadrement tout au long de l'année universitaire.

Table des matières

Remerciements	1
Introduction	5
1 Présentation du projet	6
1.1 Description de l'application	6
1.2 Besoins fonctionnels	7
1.3 Besoins non fonctionnels	7
2 Conception	8
2.1 Architecture MVC	8
2.2 Diagramme de classes UML	8
2.3 Diagramme de cas d'utilisation	9
2.4 Diagramme de séquence	10
3 Environnement de travail	11
3.1 Outils et technologies utilisés	12
3.2 Structure du projet	12
4 Répartition des tâches	14
4.1 Tâches de l'étudiant 1 – Demnati Ismail	14

4.2	Tâches de l'étudiant 2 – Salhi Amine	15
5	Explication du code	16
5.1	Connexion à la base de données – Database.java	16
5.2	Les classes modèles	16
5.3	Les classes DAO	17
5.4	Les contrôleurs	18
5.5	Les fichiers FXML	18
5.6	Fonctionnalité spécifique au projet	19
6	Interfaces de l'application	20
6.1	Fenêtre principale	20
6.2	Interfaces de gestion de l'entité 1	21
6.3	Interfaces de gestion de l'entité 2	21
6.4	Tableau de bord – Statistiques	22
6.5	Menus et dialogues	23
6.6	Export de données	23
	Conclusion	24
	Références	25

Table des figures

2.1	Diagramme de classes UML	9
2.2	Diagramme de cas d'utilisation	9
2.3	Diagramme de séquence : ajout d'un patient	10
6.1	Fenêtre principale de l'application	20
6.2	Interface de gestion des patients	21
6.3	Interface de gestion des rendez-vous	22
6.4	Tableau de bord et statistiques	23

Liste des tableaux

1.1	Besoins fonctionnels de l'application	7
3.1	Outils et technologies utilisés	12
4.1	Tâches réalisées par Demnati Ismail	14
4.2	Tâches réalisées par Salhi Amine	15
5.1	Rôle des principales classes du projet	17

Introduction

Dans le cadre du mini-projet du module Développement Java IHM, il nous a été demandé de concevoir et de développer une application de gestion complète avec une interface graphique JavaFX, connectée à une base de données MySQL et organisée selon une architecture claire.

Mon binôme a choisi le domaine hospitalier afin de réaliser une application appelée **MedVision**. Cette application permet de gérer deux entités principales : les patients et les rendez-vous médicaux. Elle propose également un tableau de bord, des fonctions de recherche et de filtrage, des statistiques et l'export des données au format CSV.

Ce rapport présente le projet selon le plan demandé : présentation générale, conception, environnement de travail, répartition des tâches, explication du code et interfaces de l'application.

Présentation du projet

1.1 Description de l'application

MedVision est une application desktop développée en JavaFX pour faciliter la gestion quotidienne d'un hôpital ou d'une clinique. Elle centralise les informations importantes liées aux patients et aux rendez-vous médicaux dans une interface graphique moderne et claire.

L'application permet à l'utilisateur de consulter un tableau de bord, d'ajouter ou modifier des patients, de planifier des rendez-vous, de filtrer les informations, de consulter des indicateurs et d'exporter les données. Le choix du domaine hospitalier est justifié par le fait qu'il permet de mettre en relation deux entités cohérentes : un patient peut avoir plusieurs rendez-vous, tandis qu'un rendez-vous appartient à un seul patient.

1.2 Besoins fonctionnels

Fonctionnalité	Description
Gestion des patients	Ajouter, modifier, supprimer, afficher et rechercher les patients.
Gestion des rendez-vous	Créer, modifier, supprimer et afficher les rendez-vous associés aux patients.
Recherche et filtrage	Rechercher par mot-clé et filtrer par sexe, statut, spécialité ou état du rendez-vous.
Tableau de bord	Afficher des indicateurs : nombre de patients, rendez-vous, statistiques et prochains rendez-vous.
Export CSV	Exporter les données affichées dans les tableaux vers un fichier CSV via FileChooser.
Messages et confirmations	Afficher des alertes pour les erreurs, confirmations et opérations réussies.
Personnalisation	Utiliser un ColorPicker pour modifier la couleur d'accent de l'interface.

TABLE 1.1 – Besoins fonctionnels de l'application

1.3 Besoins non fonctionnels

- **Ergonomie** : l'interface doit être lisible, intuitive et facile à utiliser.
- **Fiabilité** : les opérations CRUD doivent fonctionner correctement et les erreurs doivent être signalées.
- **Maintenabilité** : le projet doit être organisé en couches distinctes : modèles, vues, contrôleurs, DAO et services.
- **Sécurité minimale** : les champs doivent être contrôlés avant l'enregistrement des données.
- **Performance** : l'application doit rester fluide lors de la consultation des tableaux et du filtrage.
- **Portabilité** : le projet doit pouvoir être lancé avec Maven sur une machine disposant de Java et MySQL.

Conception

2.1 Architecture MVC

Le projet suit une organisation inspirée de l'architecture MVC. Cette séparation permet de distinguer les données, l'interface graphique et la logique de traitement. La partie **Model** contient les classes représentant les entités de l'application, notamment `Patient` et `RendezVous`. La partie **View** est composée des fichiers FXML décrivant les interfaces graphiques. La partie **Controller** contient les classes qui réagissent aux actions de l'utilisateur. La couche **DAO** assure la communication avec la base de données MySQL via JDBC.

- **Model** : classes `Patient`, `RendezVous` et autres classes métier.
- **View** : fichiers FXML tels que `dashboard.fxml`, `patients.fxml` et `rendezvous.fxml`.
- **Controller** : classes `PatientController`, `RendezVousController` et `DashboardController`.
- **DAO** : classes `Database`, `PatientDAO` et `RendezVousDAO` pour l'accès aux données.
- **Service/Repository** : couche supplémentaire utilisée pour certaines fonctionnalités avancées du tableau de bord.

2.2 Diagramme de classes UML

Le diagramme de classes suivant présente les classes principales utilisées pour gérer les patients, les rendez-vous, l'accès à la base de données et les contrôleurs.

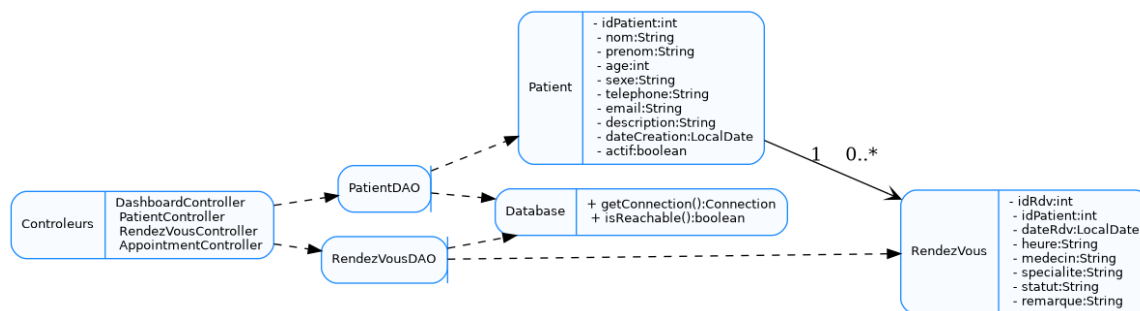


FIGURE 2.1 – Diagramme de classes UML

2.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation montre les interactions principales entre l'utilisateur de l'application et le système MedVision.

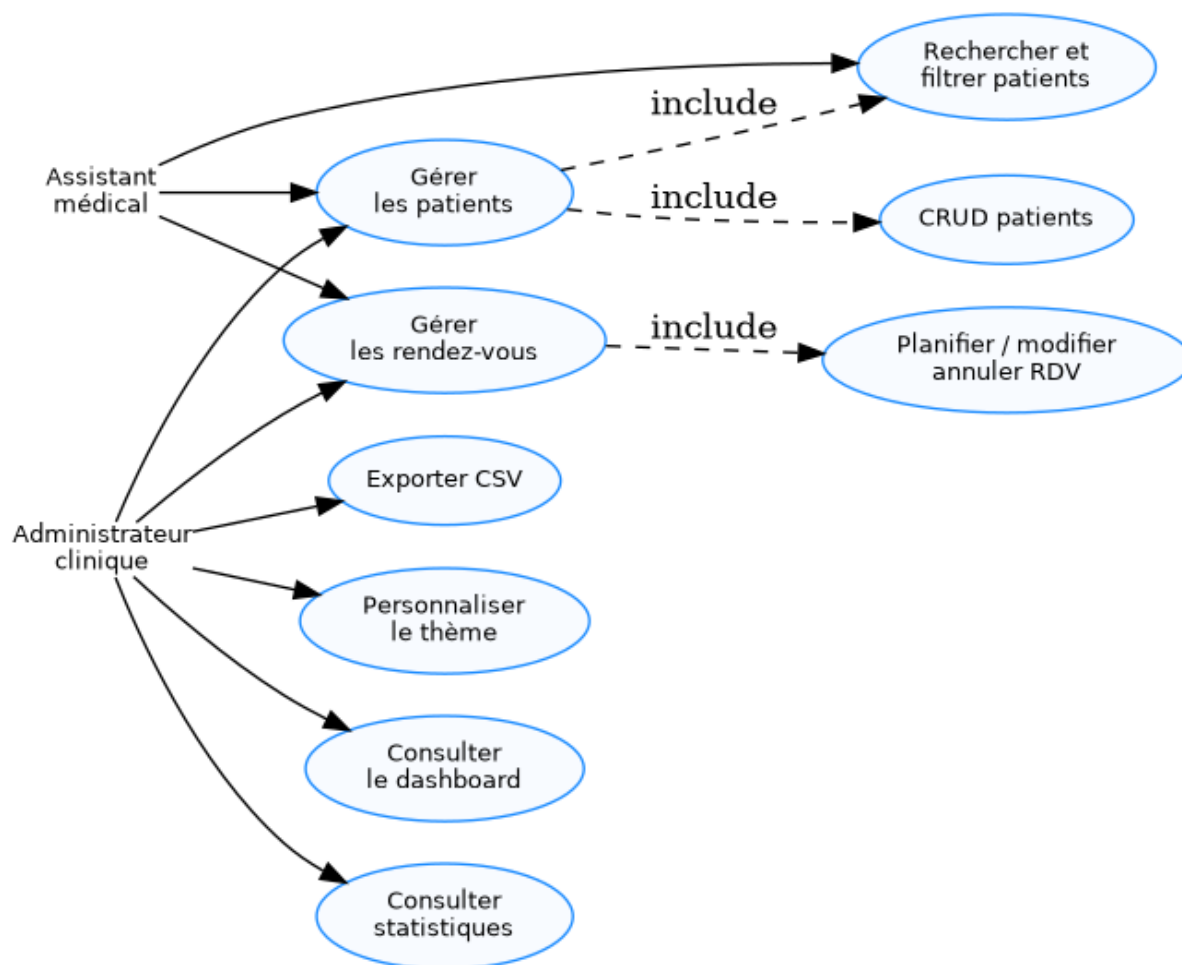


FIGURE 2.2 – Diagramme de cas d'utilisation

2.4 Diagramme de séquence

Le diagramme de séquence ci-dessous illustre le scénario d'ajout d'un patient, depuis la saisie dans l'interface jusqu'à l'enregistrement dans la base de données.

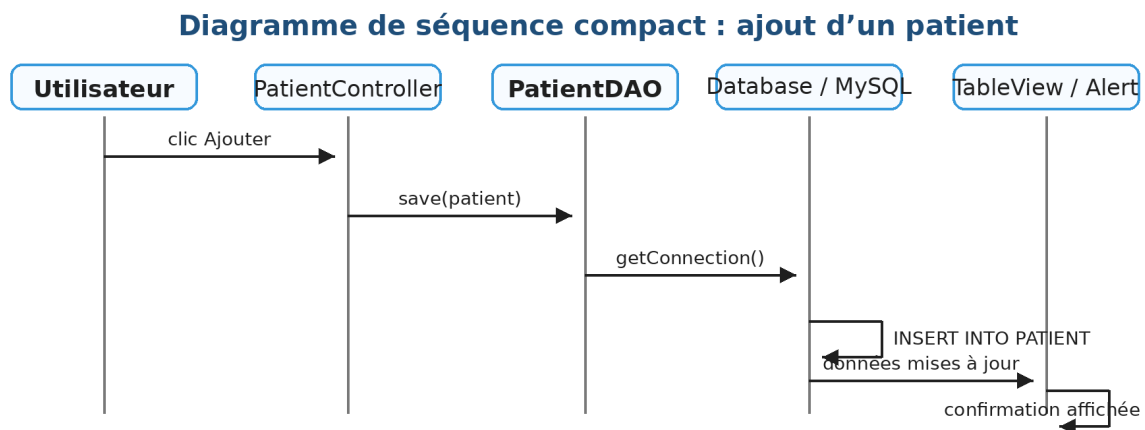


FIGURE 2.3 – *Diagramme de séquence : ajout d'un patient*

Environnement de travail

3.1 Outils et technologies utilisés

Outil / technologie	Rôle dans le projet
Java 17+	Langage principal utilisé pour développer l'application.
JavaFX	Framework utilisé pour construire l'interface graphique.
FXML	Définition déclarative des vues de l'application.
CSS	Personnalisation de l'apparence graphique.
Maven	Gestion des dépendances et lancement du projet.
MySQL	Système de gestion de base de données relationnelle.
JDBC	Connexion entre Java et MySQL.
FontAwesomeFX	Bibliothèque d'icônes utilisée dans l'interface.
Git et GitHub	Gestion de versions, dépôt distant et collaboration.
Codex	Aide au développement et à la compréhension du code.

TABLE 3.1 – Outils et technologies utilisés

3.2 Structure du projet

Le projet est organisé de manière à séparer les responsabilités : configuration, contrôleurs, modèles, accès aux données, services, vues FXML et style CSS. La structure réelle observée dans le projet est la suivante :

```
hospital-management-javafx/  
|-- README.md  
|-- init_db.sql  
|-- pom.xml  
|-- diagrams/  
|-- rapport_medvision.txt  
'-- src/  
    |-- Main.java  
    |-- config/  
    |   |-- ApplicationContext.java  
    |   '-- DatabaseConfig.java  
    |-- controller/  
    |   |-- AppointmentController.java  
    |   |-- BillingController.java  
    |   |-- DashboardController.java  
    |   '-- PatientController.java  
    |-- controllers/  
    |   |-- CsvExporter.java  
    |   |-- DashboardController.java  
    |   |-- PatientController.java  
    |   '-- RendezVousController.java  
    |-- dao/  
    |   |-- Database.java  
    |   |-- PatientDAO.java  
    |   '-- RendezVousDAO.java  
    |-- model/ et models/  
    |-- repository/  
    |-- service/  
    |-- util/  
    |-- view/ et views/  
    '-- css/style.css
```

Le fichier pom.xml montre que le projet utilise Maven avec Java 17, JavaFX 21.0.6, le connecteur MySQL JDBC 8.4.0 et FontAwesomeFX. Le dossier target est généré automatiquement par Maven et ne constitue pas un fichier source à modifier.

Répartition des tâches

4.1 Tâches de l'étudiant 1 – Demnati Ismail

Tâche réalisée	Fichier(s) concerné(s)	Statut
Organisation du travail avec GitHub	Dépôt GitHub, branches, commits, Pull Requests	Réalisé
Préparation du fichier README	README.md	Réalisé
Vérification du code, des erreurs et des bugs	src/, tests manuels de l'application, exécution Maven	Réalisé
Ajout du lien de la vidéo de démonstration	README.md	Réalisé
Préparation du rapport	rapport/rapport_medvision.pdf ou .docx	Réalisé
Vérification des livrables	README.md, init_db.sql, rapport, vidéo	Réalisé

TABLE 4.1 – Tâches réalisées par Demnati Ismail

4.2 Tâches de l'étudiant 2 – Salhi Amine

Tâche réalisée	Fichier(s) concerné(s)	Statut
Développement du code JavaFX	src/	Réalisé
Création des interfaces graphiques	src/view/, src/views/	Réalisé
Développement des classes modèles	src/model/, src/models/	Réalisé
Développement des contrôleurs	src/controller/, src/controllers/	Réalisé
Développement de l'accès aux données	src/dao/, init_db.sql	Réalisé
Ajout du style graphique	src/css/style.css	Réalisé
Réalisation de la vidéo de démonstration	Vidéo démo, lien Google Drive, présentation fonctionnelle	Réalisé

TABLE 4.2 – Tâches réalisées par Salhi Amine

Explication du code

5.1 Connexion à la base de données – Database.java

La connexion à la base de données est gérée principalement par `src/dao/Database.java`. Cette classe centralise l'ouverture de la connexion MySQL afin d'éviter la répétition du même code dans les différentes classes DAO. Le projet contient également `src/config/DatabaseConfig.java` pour regrouper les informations de configuration et `src/config/ApplicationContext.java` pour organiser certains objets partagés de l'application.

- `Database.java` ouvre la connexion JDBC vers MySQL, utilise `mysql-connector-j` et fournit une connexion aux DAO.
- `DatabaseConfig.java` regroupe les paramètres de connexion et facilite la modification de l'URL, de l'utilisateur et du mot de passe.

Grâce à cette séparation, les classes `PatientDAO` et `RendezVousDAO` peuvent accéder à la base de données sans connaître tous les détails techniques de la connexion.

5.2 Les classes modèles

Les classes modèles représentent les données manipulées par l'application. Le projet contient deux dossiers de modèles : `src/models` pour les entités principales `Patient` et `RendezVous` utilisées dans la partie CRUD liée à MySQL, et `src/model` pour les classes complémentaires de la version avancée de l'application.

Classe	Rôle
<code>models.Patient</code>	Représente l'entité Patient utilisée dans les opérations CRUD de base.
<code>models.RendezVous</code>	Représente l'entité Rendez-vous liée à un patient.
<code>model.Patient</code>	Représente un patient dans la version avancée de l'application.
<code>model.Appointment</code>	Représente un rendez-vous médical dans le module Appointment.
<code>model.Invoice</code>	Représente une facture dans le module Billing.
<code>model.DashboardMetrics</code>	Regroupe les indicateurs statistiques affichés dans le tableau de bord.
<code>model.MedicalEvent,</code> <code>NotificationItem,</code> <code>SearchResult</code>	Classes complémentaires utilisées pour les événements, notifications et résultats de recherche.

TABLE 5.1 – Rôle des principales classes du projet

5.3 Les classes DAO

Les classes DAO se trouvent dans `src/dao`. Elles isolent les requêtes SQL et permettent de garder les contrôleurs plus lisibles. Le rôle d'un DAO est d'exécuter les opérations de création, lecture, modification et suppression dans MySQL.

- `Database.java` : fournit la connexion JDBC centralisée.
- `PatientDAO.java` : gère les opérations SQL liées aux patients : ajout, modification, suppression, recherche et affichage.
- `RendezVousDAO.java` : gère les opérations SQL liées aux rendez-vous et leur association avec un patient.
- Le dossier repository contient aussi des implémentations en mémoire pour certains modules avancés : `InMemoryPatientRepository`, `InMemoryAppointmentRepository`

et `InMemoryInvoiceRepository`.

5.4 Les contrôleurs

Les contrôleurs assurent la communication entre les vues FXML et le reste de l'application. Le projet contient deux dossiers de contrôleurs, ce qui montre l'existence d'une partie CRUD principale et d'une partie interface avancée.

- `src/controllers/PatientController.java` : gère la saisie, la modification, la suppression, la recherche et l'affichage des patients.
- `src/controllers/RendezVousController.java` : gère la saisie, la modification, la suppression et l'affichage des rendez-vous.
- `src/controllers/DashboardController.java` : gère le tableau de bord dans la partie CRUD principale.
- `src/controllers/CsvExporter.java` : permet l'export des données vers un fichier CSV.
- `src/controller/AppointmentController.java` : gère l'interface avancée des rendez-vous.
- `src/controller/BillingController.java` : gère le module de facturation.
- `src/controller/DashboardController.java` et `src/controller/PatientController.java` : gèrent les vues avancées du tableau de bord et des patients.

5.5 Les fichiers FXML

Les fichiers FXML décrivent la structure de l'interface graphique. Ils permettent de séparer la partie visuelle du code Java. Dans ce projet, les vues sont réparties entre `src/view` et `src/views`.

- `src/view/dashboard.fxml` : tableau de bord avancé de l'application.
- `src/view/patients.fxml` : interface avancée de gestion des patients.
- `src/view/appointments.fxml` : interface avancée de gestion des rendez-vous.
- `src/view/billing.fxml` : interface de facturation.
- `src/views/dashboard.fxml` : tableau de bord de la partie CRUD principale.
- `src/views/patients.fxml` : interface principale de gestion des patients.
- `src/views/rendezvous.fxml` : interface principale de gestion des rendez-vous.

- `src/css/style.css` : feuille de style qui personnalise l'apparence de l'application.

5.6 Fonctionnalité spécifique au projet

Une fonctionnalité importante du projet est l'export des données au format CSV. Le projet contient deux classes utiles à cet effet : `src/controllers/CsvExporter.java` et `src/util/CsvExporter.java`. Cette fonctionnalité utilise le choix d'un emplacement de sauvegarde et transforme les données affichées dans les tableaux en fichier exploitable hors de l'application.

Le projet contient aussi des services qui améliorent l'expérience utilisateur : `DashboardService` pour les indicateurs, `NotificationService` pour les notifications, `GlobalSearchService` pour la recherche globale, `BillingService` pour la facturation, `AppointmentService` pour les rendez-vous et `AiSimulationService` pour une simulation intelligente dans la partie avancée. Ces services rendent l'application plus complète qu'un simple CRUD.

Interfaces de l'application

6.1 Fenêtre principale

La fenêtre principale de MedVision sert de point d'entrée de l'application. Elle contient la navigation vers les différentes sections : tableau de bord, patients, rendez-vous, facturation et autres vues disponibles.



FIGURE 6.1 – Fenêtre principale de l'application

6.2 Interfaces de gestion de l'entité 1

La première entité principale est **Patient**. L'interface de gestion des patients contient un formulaire de saisie, des boutons d'action, des filtres et un tableau récapitulatif. L'utilisateur peut ajouter, modifier, supprimer, rechercher et exporter les patients.

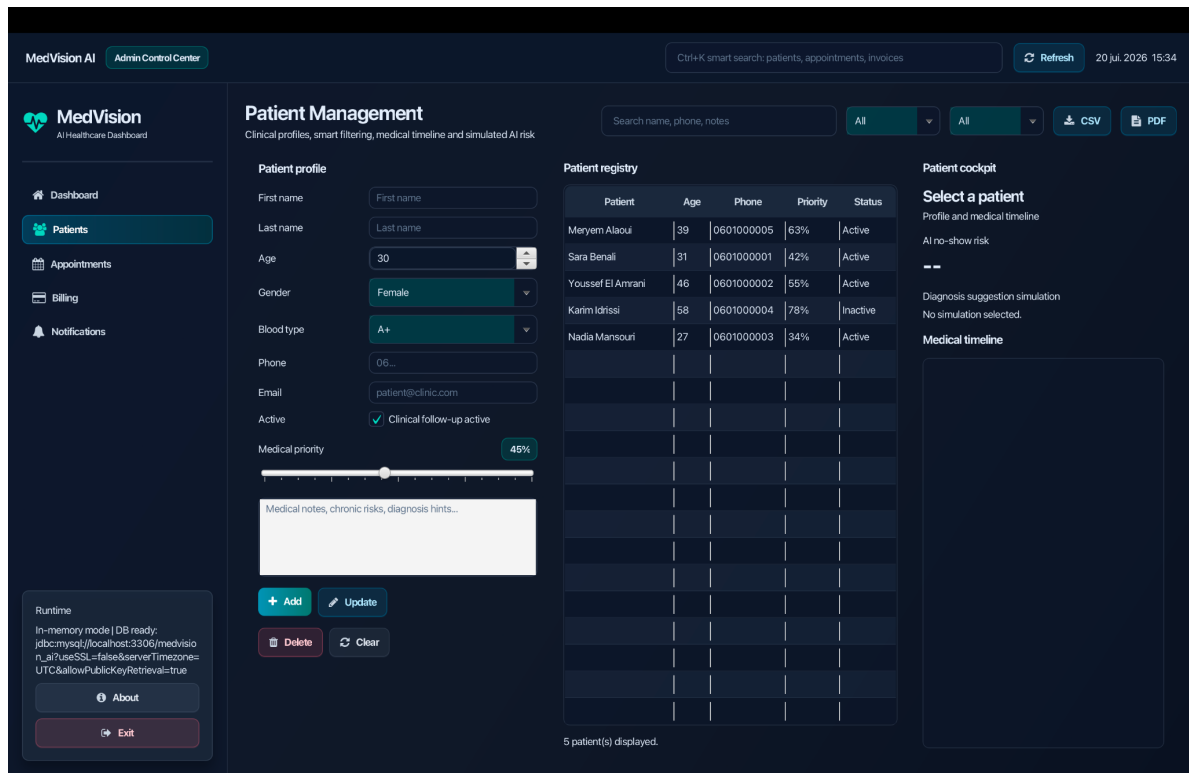


FIGURE 6.2 – *Interface de gestion des patients*

6.3 Interfaces de gestion de l'entité 2

La deuxième entité principale est **Rendez-vous**. Cette interface permet d'associer un rendez-vous à un patient, de choisir une date, une heure, un médecin, une spécialité et un statut. Elle permet également de filtrer les rendez-vous et de les exporter.

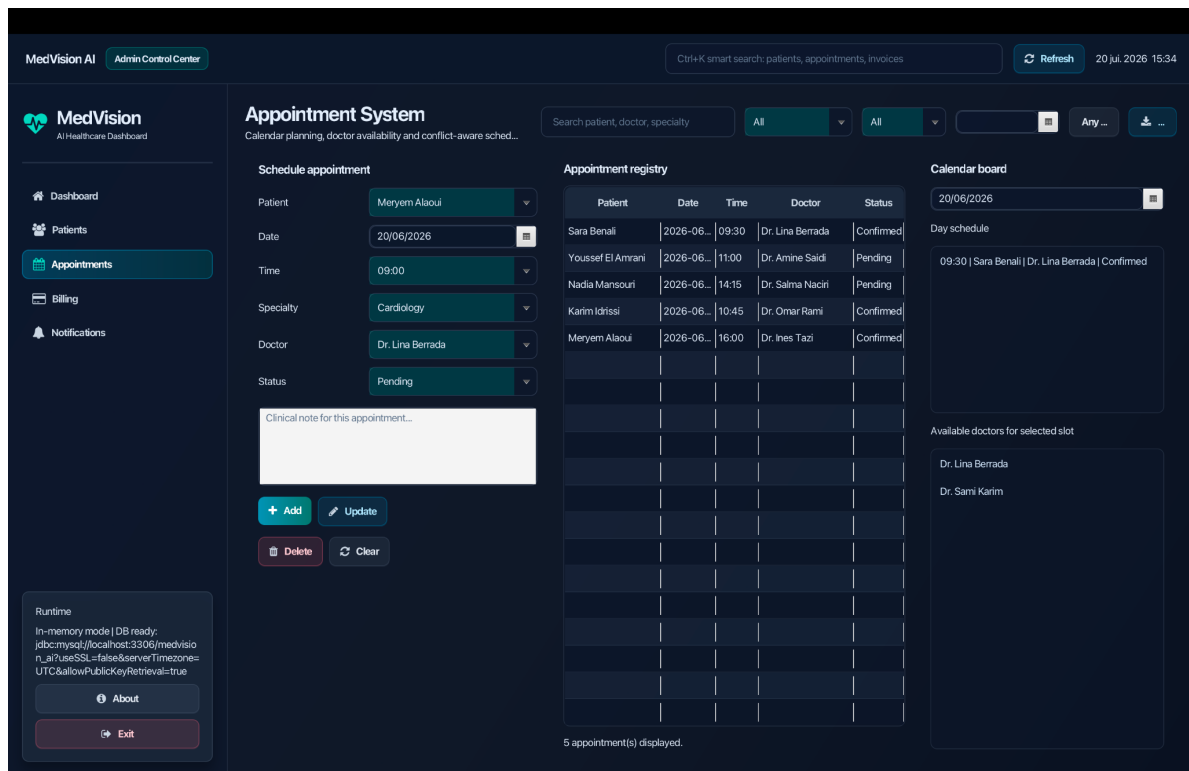


FIGURE 6.3 – *Interface de gestion des rendez-vous*

6.4 Tableau de bord – Statistiques

Le tableau de bord présente des indicateurs chiffrés, des graphiques et des listes utiles pour suivre l'activité de l'hôpital. Il permet notamment de visualiser le nombre total de patients, les rendez-vous du jour, les patients actifs et certains indicateurs de suivi.

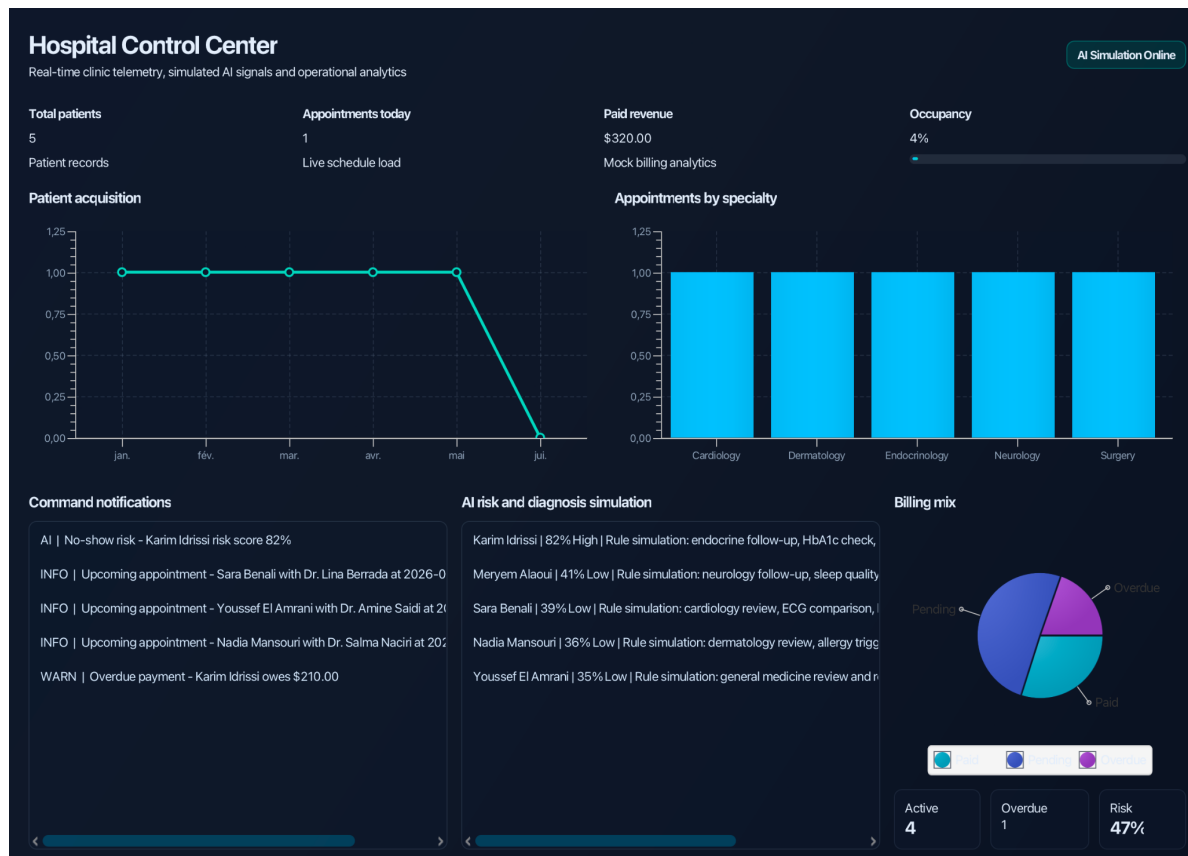


FIGURE 6.4 – Tableau de bord et statistiques

6.5 Menus et dialogues

L'application utilise des menus, des boutons, des Tooltips et des boîtes de dialogue. Les Alert servent à informer l'utilisateur en cas de succès, d'erreur ou de confirmation avant une action sensible comme la suppression.

- Alert de confirmation avant suppression.
- Alert d'erreur en cas de champ vide ou invalide.
- Tooltip pour guider l'utilisateur sur certains champs.
- MenuBar et boutons de navigation pour structurer les actions.

6.6 Export de données

L'export CSV permet de récupérer les données affichées dans les tableaux pour les consulter en dehors de l'application. Cette fonctionnalité répond à l'exigence du mini-projet concernant l'export des données via FileChooser.

Conclusion

Ce mini-projet nous a permis de mettre en pratique plusieurs notions importantes du développement d'interfaces graphiques avec JavaFX. À travers l'application MedVision, nous avons réalisé une solution de gestion d'hôpital permettant de manipuler des patients, des rendez-vous, des statistiques et des exports de données.

Le projet respecte les principales exigences demandées : utilisation de JavaFX, organisation MVC, fichiers FXML et CSS, base de données MySQL avec script SQL, opérations CRUD, recherche, filtrage, statistiques, export CSV, README et vidéo de démonstration.

Sur le plan technique, ce travail nous a également permis de découvrir l'importance de GitHub pour le travail en binôme, la gestion des versions et la préparation des livrables.

Enfin, ce projet constitue une base qui pourrait être améliorée par l'ajout d'une authentification, d'une gestion plus avancée des rôles, d'une historisation médicale et d'une connexion plus complète entre tous les modules métier.

Références

- Documentation officielle JavaFX : <https://openjfx.io>
- Documentation MySQL : <https://dev.mysql.com/doc/>
- Documentation Maven : <https://maven.apache.org/>
- Documentation GitHub : <https://docs.github.com/>
- Dépôt GitHub du projet : <https://github.com/ismd2005-lab/git-commit--m-Initial-commit-hospital-JavaFX-project-.git>
- Vidéo de démonstration Google Drive : https://drive.google.com/file/d/1MH-9QFIjpN1XuioeSiGLIeEG2_jH228b/view?usp=sharing